

Rings: A peer-to-peer network for the sovereign age

Ryan J. Kung
ryankung@ieee.org

Created: February 8, 2023
Source: RingsNetwork/whitepaper repository
Timestamp: 2023-02-08T13:27:55Z
Last updated: July 4, 2026

Abstract

Rings Network is specified as a peer-to-peer protocol family over the circular carrier $R = \mathbb{Z}/2^{160}\mathbb{Z}$. The overlay layer is a Correct-Chord maintenance object; DID, VID, mailbox, hidden-service, ranking, and sequencing protocols are namespace objects over R ; placement is post-composition by the live owner map; effectful transitions are Kleisli morphisms of a writer monad over protocol actions. The paper states the carrier, topology, category, state, effect, cryptographic, verification, and evaluation obligations of the target Rings model while keeping implementation-facing algorithms, figures, and performance baselines as executable witnesses of the model.

1 Formal Model

Assumption 1 (System scope). *Rings is modeled as a fail-stop structured overlay with authenticated participants, signed protocol messages, asynchronous delivery, and runtime-specific IO interpreters. Byzantine consensus is not a base-overlay operation. Stronger ordering is represented by side-car witnesses. Cryptographic multiplication, threshold interpolation, pairings, and subgroup checks live in explicit cryptographic carriers, not in the Chord identifier carrier.*

Definition 1 (Identifier carrier). *Let*

$$M = 2^{160}, \quad R = \mathbb{Z}/M\mathbb{Z}.$$

For $a, x \in R$, define clockwise distance

$$\delta_a(x) = (x - a) \bmod M$$

and circular interval

$$(a, b]_R = \{x \in R : 0 < \delta_a(x) \leq \delta_a(b)\}.$$

The overlay uses $(R, +, 0, -)$ as an additive cyclic group. Multiplication in R is outside the routing model.

Definition 2 (Live topology). *For a nonempty live set $N \subset R$ and $n, k \in R$,*

$$\text{owner}_N(k) = \arg \min_{u \in N} \delta_k(u),$$

$$\text{pred}_N(n) = \arg \min_{p \in N \setminus \{n\}} \delta_p(n),$$

$$\text{Succ}_N^q(n) = \text{take}_q(\text{sort}_{\delta_n}(N \setminus \{n\})).$$

Argmins are unique because δ_a is injective on N .

Assumption 2 (Correct-Chord stable base). *Let r be the successor-list length. Steady-state algebra assumes*

$$|N| \geq r + 1.$$

Bootstrap states with fewer members are initialization states. The maintenance reference is Zave's Correct Chord transition family of join, rectify, and stabilize [1].

Definition 3 (Replica set). *For replication factor r ,*

$$\text{Rep}_N^r(k) = \{\text{owner}_N(k)\} \cup \text{set}(\text{Succ}_N^{r-1}(\text{owner}_N(k))).$$

Thus the owner is included at index 0 of storage responsibility.

Definition 4 (Rings overlay object). *A steady Rings overlay state over N is*

$$\mathcal{R}_N = (R, N, \text{owner}_N, \text{pred}_N, \text{Succ}_N^r, \text{Rep}_N^r, F, E),$$

where F is the sparse finger relation and E is the replicated entry algebra. The safety root is $(\text{pred}_N, \text{Succ}_N^r)$; fingers are routing witnesses.

Proposition 1 (Rotation equivariance). *For $t \in R$, $N + t = \{n + t : n \in N\}$ satisfies*

$$\text{owner}_{N+t}(k+t) = \text{owner}_N(k) + t.$$

Proof. For each $n \in N$,

$$\delta_{k+t}(n+t) = ((n+t) - (k+t)) \bmod M = \delta_k(n).$$

Translation preserves the order used by $\arg \min$. $\square$

Constraint 1 (Carrier separation). *Overlay placement may use only*

$$0, +, -, \delta_a, (a, b]_R, \text{owner}_N, \text{Rep}_N^r.$$

A protocol requiring multiplication, inversion, scalar multiplication, polynomial interpolation, pairings, or subgroup checks names a separate carrier such as a finite field $\mathbb{F}_q$ or elliptic-curve group $\mathbb{G}$.

Object	Carrier or law
Chord IDs	additive cyclic group R
DID proof	signature verifier and transcript law
VID placement	$H_\nu : X_\nu \rightarrow R$ then owner map
Storage merge	join semilattice or explicit finalizer
E2E encryption	finite field or curve group
Sequencing	partial order over witness domains
Effects	writer monad over action lists

Table 1: Carrier boundaries in the Rings model

2 Architecture As Interfaces

Definition 5 (Layer tuple). *The target architecture is a six-tuple*

$$\mathcal{A} = (I, T, O, X, P, L)$$

where I is identity, T is transport, O is overlay, X is extension runtime, P is protocol algebra, and L is application logic.

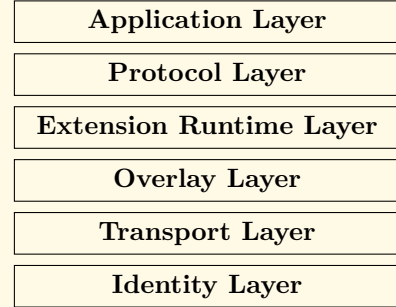


Figure 1: Layers of Rings Network

Definition 6 (Identity proof). *A DID proof family is a tuple*

$$\Pi = (K, \Sigma, V, D, \text{enc}, \text{ttl})$$

where K is the public-key space, Σ is the signature space, $V : K \times \Sigma \times B^ \rightarrow \{\top, \perp\}$ is verification, $D : K \rightarrow R$ is DID derivation, enc is signature encoding, and ttl bounds replay.*

Invariant 1 (Session authority). For account DID d and delegated session key s ,

$$\text{Session}(d, s, e) \Rightarrow V(k_d, \sigma_{d \rightarrow s}, d \| s \| e) = \top \wedge \text{now} < e.$$

Every hop may check session authority without resolving a W3C DID document in the overlay path.

Definition 7 (Transport interface). Transport is modeled as

$$T = (C, \text{offer}, \text{answer}, \text{send}, \text{recv})$$

with channel state C and action alphabet

$$A_T = \{\text{Offer}, \text{Answer}, \text{Ice}, \text{Open}, \text{Send}, \text{Recv}, \text{Close}\}.$$

WebRTC and native transports are interpreters of A_T^* .

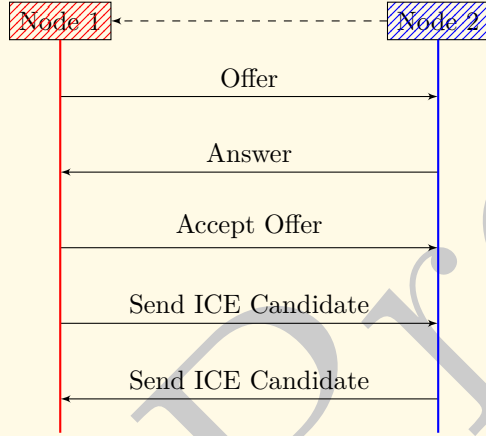


Figure 2: Standard SDP/ICE exchange

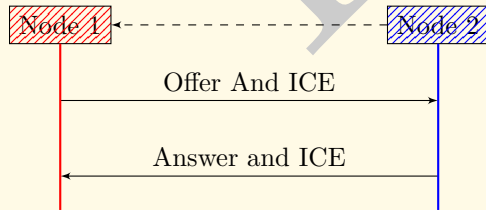


Figure 3: Rings single round-trip SDP/ICE exchange

Definition 8 (Runtime interpreter). An extension runtime is

$$\mathcal{X} = (S_X, A_X, \text{cap}, I_X),$$

where S_X is extension state, A_X is the action alphabet, $\text{cap} \subseteq A_X$ is the capability set, and

$$I_X : A_X^* \rightarrow \text{IO}$$

is a monoid homomorphism into IO actions under sequential composition.

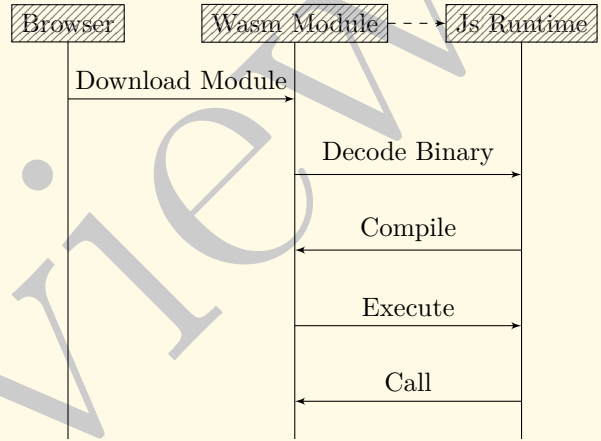


Figure 4: Wasm module execution boundary

Obligation 1 (Runtime confinement). For every extension transition τ and interpreter I_X ,

$$\tau : S \times I \rightarrow \mathcal{E} + (S \times O \times A_X^*)$$

may emit only actions in cap . The interpreter rejects $a \notin \text{cap}$ before IO.

3 Category and Effects

Definition 9 (Namespace object). A namespace ν is an object of the slice category $\mathbf{Set}/R$:

$$H_\nu : X_\nu \rightarrow R.$$

DIDs, VIDs, mailbox roots, service descriptors, sub-ring anchors, and ranking keys are distinct namespace objects over the same carrier.

Definition 10 (Placement functor). *For live topology N , define*

$$\Omega_N : \mathbf{Set}/R \rightarrow \mathbf{Set}/N$$

by

$$\Omega_N(X, H) = (X, \text{owner}_N \circ H).$$

The placement map of namespace ν is

$$P_\nu^N = \text{owner}_N \circ H_\nu : X_\nu \rightarrow N.$$

Proposition 2 (Placement functoriality). *If $f : X_\nu \rightarrow X_\mu$ satisfies $H_\mu \circ f = H_\nu$, then*

$$P_\mu^N \circ f = P_\nu^N.$$

Proof. Post-compose $H_\mu \circ f = H_\nu$ with owner_N . $\square$

Definition 11 (Writer effect monad). *For action alphabet A , let A^* be the free monoid of finite action lists. Define*

$$\mathsf{T}_A(X) = X \times A^*,$$

$$\eta(x) = (x, \epsilon),$$

$$(x, \alpha) \gg= f = \text{let } f(x) = (y, \beta) \text{ in } (y, \alpha \cdot \beta).$$

Proposition 3 (Effect associativity). *Kleisli composition for T_A is associative.*

Proof. The accumulated effect is $\alpha \cdot \beta \cdot \gamma$; associativity follows from associativity of the free monoid A^* . $\square$

Definition 12 (Typed failure stack). *Rings uses strict error-over-writer composition:*

$$\mathsf{T}_A^\mathcal{E}(X) = \mathcal{E} + (X \times A^*).$$

An error branch has no executable action list. Recoverable reports are ordinary successful actions.

Definition 13 (Protocol object). *A protocol namespace is*

$$\mathcal{P}_\nu = (X_\nu, S_\nu, I_\nu, O_\nu, A_\nu, \mathcal{E}_\nu, H_\nu, \tau_\nu)$$

with transition

$$\tau_\nu : S_\nu \times I_\nu \rightarrow \mathsf{T}_{A_\nu}^{\mathcal{E}_\nu}(S_\nu \times O_\nu).$$

Definition 14 (Protocol morphism). *A morphism $\Sigma : \mathcal{P}_\nu \rightarrow \mathcal{P}_\mu$ is a family*

$$(\sigma_X, \sigma_S, \sigma_I, \sigma_O, \sigma_\mathcal{E}, \sigma_A^*)$$

where $\sigma_A^* : A_\nu^* \rightarrow A_\mu^*$ is a monoid homomorphism and

$$H_\mu \circ \sigma_X = H_\nu,$$

$$\mathsf{T}_{\sigma_A^*}^{\sigma_\mathcal{E}}(\sigma_S \times \sigma_O) \circ \tau_\nu = \tau_\mu \circ (\sigma_S \times \sigma_I).$$

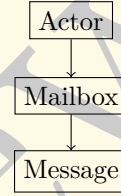


Figure 5: Actor Model

4 Chord Overlay

Rings builds on Chord [2] and adopts Correct Chord [1] as the maintenance reference. The formal object is $\mathcal{R}_N$; the algorithms below are transition witnesses.

Definition 15 (Local node state). *For node $n \in N$,*

$$S_n = (p_n, L_n, F_n, E_n)$$

where p_n is predecessor, L_n is successor list, F_n is sparse finger table, and E_n is local entry evidence. The fixpoint target is

$$p_n = \text{pred}_N(n), \quad L_n = \text{Succ}_N^r(n).$$

Definition 16 (Finger target). *For slot i ,*

$$\text{target}_i(n) = n + 2^i,$$

$$\text{finger}_i(n) = \text{owner}_N(\text{target}_i(n)).$$

Sparse no-wrap state permits $F_n(i) = \perp$; present fingers are performance witnesses.

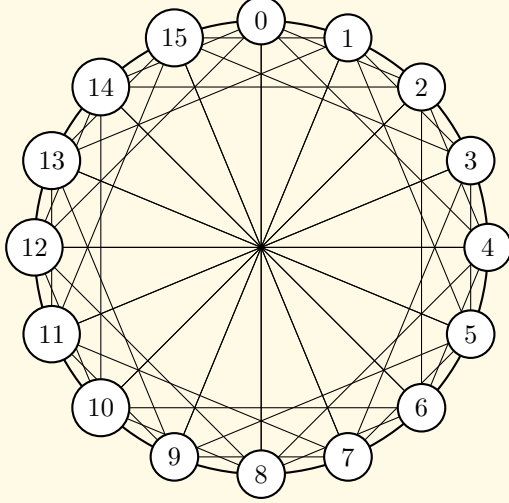


Figure 6: Chord algorithm, lookup protocol

Invariant 2 (Ring fixpoint).

$$\text{Inv}_{\text{ring}}(N, S) \equiv \forall n \in N. p_n = \text{pred}_N(n) \wedge L_n = \text{Succ}_N^r(n).$$

Invariant 3 (Lookup owner preservation). For steady topology N ,

$$\text{Lookup}(S, k) \Downarrow \text{owner}_N(k).$$

The judgment $\Downarrow$ is reachability in the action-interpretation transition relation generated by Algorithms 2 and 3.

5 DID and Resource Algebra

Definition 17 (DID namespace). A DID namespace is $H_D : X_D \rightarrow R$. Account-controlled DIDs add proof family Π and session relation $\text{Session}(d, s, e)$.

Definition 18 (VID namespace). A VID namespace is $H_V : X_V \rightarrow R$ with no private key attached to $H_V(x)$. Authorization is carried by entry operations:

$$\text{Auth}_V(\text{op}, x, \sigma, \text{caps}) \in \{\top, \perp\}.$$

Algorithm 1 DHT Join Transitions

```

1: function BEGINJOIN( $n, \text{known}$ )
2:    $\text{pred}[n] \leftarrow \text{none}$ 
3:    $a \leftarrow \text{FindSuccessor}(n.\text{did})$ 
4:   return  $\text{Remote}(\text{known}, a)$ 
5: function INSTALLSUCCESSOR( $n, s$ )
6:    $\text{succ}[n] \leftarrow \text{take}(r, [s])$ 
7:   return  $\text{Remote}(s, \text{QuerySuccList}(s))$ 
8: function INSTALLSUCCLIST( $n, s, \text{list}$ )
9:    $\text{succ}[n] \leftarrow \text{take}(r, s :: \text{list})$ 
10:  if  $\text{head}(\text{succ}[n]) \neq \text{none}$  then
11:     $h \leftarrow \text{head}(\text{succ}[n])$ 
12:     $\text{notify} \leftarrow \text{Notify}(h, n.\text{did})$ 
13:     $\text{sync} \leftarrow \text{SyncStorage}(h)$ 
14:    return  $\text{Multi}(\text{notify}, \text{sync})$ 
15:  else
16:    return  $\text{Noop}$ 

```

Definition 19 (Biased order). For fixed point C ,

$$A \leq_C B \iff \delta_C(A) \leq \delta_C(B).$$

Definition 20 (Replicated entry algebra). For each VID v , entry state is either a join semilattice

$$(E_v, \sqsubseteq, \sqcup, \perp)$$

with commutativity, associativity, idempotence, and bottom identity, or a pair

$$(E_v, \text{conflict}_v, \text{finalize}_v)$$

that resolves conflicts explicitly. The semilattice case follows CRDT merge discipline [3].

Invariant 4 (Placement).

$$\text{Inv}_{\text{place}} \equiv \forall \nu, x. \text{replicas}(H_\nu(x)) \subseteq \text{Rep}_N^r(H_\nu(x)).$$

Definition 21 (Mailbox). For DID d and epoch e ,

$$\text{mbox}(d, e) = H_M(\text{rings.mailbox} \| d \| e).$$

Mailbox entries are encrypted envelopes with nonce, expiration, sender proof, recipient DID, and ciphertext.

Algorithm 2 DHT Lookup Algorithm

```
1: function LOOKUP(key, node)
2:   s ← head(succ[node])
3:   if s = none then
4:     return Repair(QuerySuccList(node))
5:   if key ∈ (node.did, s] then
6:     return Local(s)
7:   next ← closest_preceding_node(node, key)
8:   if next = none then
9:     return Repair(QuerySuccList(s))
10:  else
11:    a ← FindSuccessor(key)
12:    return Remote(next, a)
```

Algorithm 3 DHT Stabilization

```
1: procedure STABILIZE(n, topo)
2:   actions ← []
3:   c ← topo.pred
4:   s ← head(succ[n])
5:   if c ≠ none and c ∈ (n.did, s) then
6:     succ[n] ← insert(succ[n], c)
7:     actions ← actions :: FindSuccessor(c)
8:   succ[n] ← take(r, merge(succ[n], topo.succ))
9:   if head(succ[n]) ≠ none then
10:    h ← head(succ[n])
11:    actions ← actions :: Notify(h, n.did)
12:  return Multi(actions)
```

Definition 22 (Hidden service descriptor). A service descriptor is

$$h = (\nu, \rho, \text{Intro}, \text{Relay}, \text{Cap}, K, \text{exp}, \text{rank}).$$

Its VID is $H_H(h)$; lookup returns placement, while authorization is $\text{Auth}_H(h, \sigma)$.

Definition 23 (Subring). A subring is

$$\mathcal{S}_u = (u, N_u, \text{owner}_{N_u}, \text{Succ}_{N_u}^{r_u}, E_u)$$

where $u = H_U(\text{policy})$ anchors membership. Membership operations form a mergeable log or finalized state.

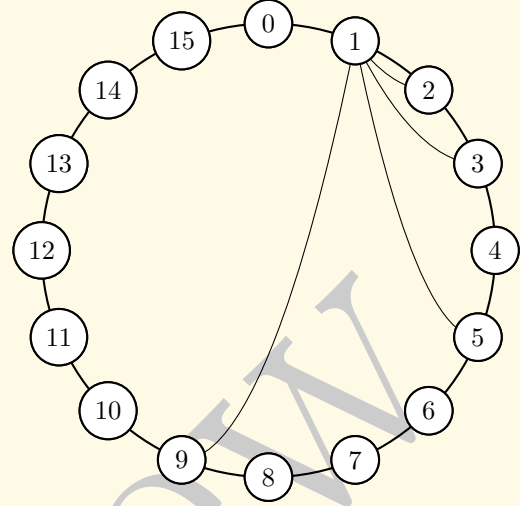


Figure 7: Chord algorithm, lookup protocol

Algorithm 4 Biased Circular Comparison

```
1: function BIASEDCOMPARE(A, B, C)
2:   a ← δC(A)
3:   b ← δC(B)
4:   if a < b then
5:     return A <C B
6:   else if a > b then
7:     return B <C A
8:   else
9:     return A = B
```

6 Cryptographic Carriers and Traffic

Definition 24 (End-to-end carrier). Let $\mathbb{G}$ be an elliptic-curve group of order q with scalar field $\mathbb{Z}_q$. Private key $x \in \mathbb{Z}_q$ has public key $h = g^x$. This carrier is distinct from R .

Definition 25 (Signed encrypted frame). An encrypted frame is

$$f = (\text{sid}, \text{seq}, \text{last}, \text{cipher}, \text{pk}_s, \sigma_s).$$

Algorithm 5 ElGamal Encryption and Decryption

```

1: Input: Message  $m$ , private key  $x \in \mathbb{Z}_q$ , public
   key  $h = g^x$ , generator  $g$ 
2: Output: Encrypted message  $(c_1, c_2)$ 
3: procedure ENCRYPTION
4:   Choose a random integer  $k \in \mathbb{Z}_q$ 
5:   Compute  $c_1 = g^k$ 
6:   Compute  $c_2 = m \cdot h^k$ 
7:   return  $(c_1, c_2)$ 
8: procedure DECRYPTION
9:   Compute  $m = c_2 \cdot (c_1)^{-x}$ 
10:  return  $m$ 

```

Validity is

$$V(pk_s, \sigma_s, sid || seq || last || cipher) = \top \\ \wedge \text{Session}(D(pk_s), sid, e).$$

Definition 26 (Secret sharing placement). For threshold polynomial $P \in \mathbb{F}_q[X]$, shares are

$$sh_i = (i, P(i)) \in \mathbb{F}_q^2.$$

Overlay placement is $H_S(sh_i) \in R$; reconstruction is field interpolation in $\mathbb{F}_q$ [4].

Definition 27 (Relay frame). A hop-by-hop frame is

$$r = (\text{path}, \text{dst}, \nu, \text{session}, \text{payload}, \text{report}).$$

Rings borrows message splitting vocabulary from MSRP [5], while relay validity is native to this tuple.

Invariant 5 (Chunk reassembly). For message m with chunks c_i ,

$$\text{complete}(m) \Leftrightarrow \forall i < \text{total}(m). \exists! c_i \\ \wedge \sum_i |c_i| \leq B_m \\ \wedge \text{now} < \text{ttl}(m).$$

Algorithm 6 End-to-End Chunking Algorithm

```

1: procedure E2E CHUNKING
2:   Input: data, chunk size
3:   Output: chunks
4:   chunks  $\leftarrow []$ 
5:   total-chunks  $\leftarrow \text{ceil}(\text{len}(\text{data})/\text{chunk-size})$ 
6:   for  $i$  in range(total-chunks) do
7:     chunk  $\leftarrow \text{data}[i*\text{chunk-size} : (i+1)*\text{chunk-size}]$ 
8:     append chunk to chunks
9:   return chunks

```

7 Ordering, Ranking, and Security

Definition 28 (Sidecar witness). A witness is

$$w = (\text{domain}, \text{root}, \text{height}, \text{time}, \text{finality}).$$

For message commitment $c = H(\text{msg})$,

$$\text{Witness}(\text{msg}, w) \Leftrightarrow \text{Include}(c, w.\text{root}) = \top \\ \wedge \text{Final}(w) = \top.$$

Definition 29 (Witness order). For messages a, b with witnesses w_a, w_b ,

$$a \prec b \Leftrightarrow \text{domain}(w_a) = \text{domain}(w_b) \\ \wedge \text{time}(w_a) < \text{time}(w_b).$$

Different domains require an application comparison morphism

$$\chi : \text{Domain}_a \times \text{Domain}_b \rightarrow \text{Order}.$$

Definition 30 (Ranking event). A ranking event is

$$q = (\text{subject}, \text{rater}, \text{behavior}, \\ \text{evidence}, \text{epoch}, \text{weight}, \sigma).$$

Its validity predicate is

$$m_q = \text{subject} || \text{behavior} || \text{evidence} || \text{epoch}, \\ \text{RankValid}(q) \Leftrightarrow V(k_{\text{rater}}, \sigma, m_q) = \top.$$

Definition 31 (Score aggregation). For epoch e and subject s ,

$$\text{score}_e(s) = \text{robust}\{\text{weight}(q) \cdot \text{value}(q) : q.\text{subject} = s\}.$$

The aggregation function is median, trimmed mean, or another declared robust statistic.

Attack	Predicate	Rings control
Sybil	many DIDs, one actor	admission cost, ranking, challenge
Replay	stale valid σ	nonce, TTL, session epoch
MITM	wrong key for DID	DID proof, signed envelope, E2E
DoS	unbounded resource use	quotas, ranking, chunk bounds

Table 2: Security predicates and controls

Obligation 2 (Replay rejection). For session s and message nonce η ,

$$\text{accept}(s, \eta, t) \Rightarrow \eta \notin \text{Seen}_s \wedge t < \text{exp}(s).$$

The transition inserts η into Seen_s before emitting delivery actions.

8 Verification and Evaluation

Definition 32 (State relation). A model configuration is

$$C = (S, Q)$$

where S is protocol state and $Q \in A^*$ is the pending action list. The step relation

$$C \rightarrow C'$$

interprets the head of Q , applies a pure transition τ , and appends emitted actions. This is the TLA-style state relation used for finite model exploration [6, 7].

Invariant 6 (Effect refinement). For implementation path Impl_A and action interpreter $I_A^\sharp$,

$$\text{Impl}_A(s, i) \preceq I_A^\sharp(\tau(s, i)).$$

Runtime scheduling and logging are refinement details; protocol state changes factor through τ .

Obligation 3 (Topology coverage). The model-checking domain contains representative layouts

$$\mathcal{L} = \{\text{line}, \text{wrap}, \text{gap}, \text{partition}, \text{merge}, \text{churn}\}$$

and schedule classes

$$\mathcal{S} = \{\text{loss}, \text{dup}, \text{reorder}, \text{timeout}, \text{restart}\}.$$

Each checked property names (L, S, P) with $L \in \mathcal{L}$, $S \in \mathcal{S}$, and predicate P .

Definition 33 (Latency decomposition). End-to-end latency is modeled as

$$T_{\text{total}} = T_{\text{conn}} + h \cdot T_{\text{hop}} + T_{\text{crypto}} + T_{\text{app}} + T_{\text{witness}}.$$

For Chord lookup, $h = O(\log |N|)$ under populated fingers.

Fraction of failed nodes	Mean routing hops	Mean num. of lookup timeouts
0	3.84	0.0
0.1	4.03	0.60
0.2	4.22	1.17

Table 3: Latency on failed node rate of Chord [2]

Definition 34 (Reliability baseline). For lookup trials Q and failed lookups F ,

$$\text{failrate} = \frac{|F|}{|Q|}.$$

For timeout count $\text{to}(q)$,

$$\text{timeoutMean} = \frac{1}{|Q|} \sum_{q \in Q} \text{to}(q).$$

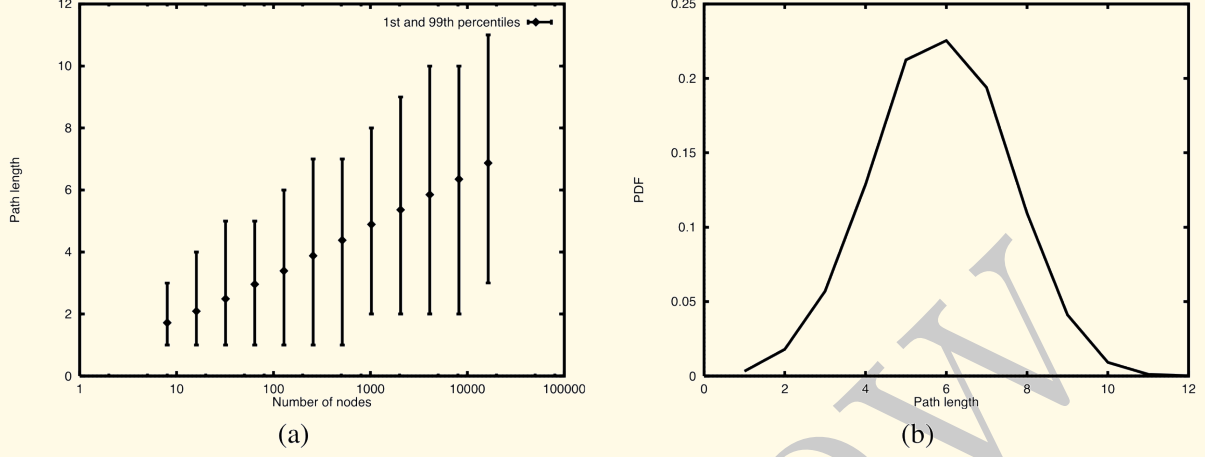


Figure 8: Chord lookup path [2]

join/leave rate (per sec./stab. period)	Mean routing hops	Mean num. of lookup timeouts	Lookup failures per 10k
0.05/1.5	3.90	0.5	0
0.10/2	3.83	0.11	0
0.15/4.5	3.84	0.16	2
0.20/6	3.81	0.23	5
0.25/7.5	3.83	0.30	6
0.30/9	3.91	0.34	8
0.35/10.5	3.94	0.42	16
0.40/12	4.06	0.46	15

Table 4: Failure rate of Chord [2]

Obligation 4 (Evaluation morphism). *Chord reference metrics instantiate Rings evaluation by*

$$\begin{aligned} \Phi_{eval} : (h, timeouts, failures)_{Chord} \\ \mapsto (T_{total}, timeoutMean, failrate)_{Rings}, \end{aligned}$$

where Φ_{eval} adds transport establishment, cryptographic work, witness verification, and application protocol cost.

9 Related Work

Kademlia-derived networks use XOR distance rather than circular successor order [8–10]. Tor and Nym emphasize relay anonymity rather than structured ownership [11, 12]. Rings selects Chord because the functions $owner_N$, $Succ_N^r$, and Rep_N^r give DID and VID placement a small algebraic core.

10 Conclusion

Rings is the tuple

$$\mathcal{R} \setminus \setminus f = (R, \mathcal{R}_N, \mathbf{Set}/R, \Omega_N, \mathcal{T}_A^\mathcal{E}, \Pi, \mathbb{G}, \mathcal{X}).$$

Its base overlay is Correct Chord over an additive circular carrier. Its resources are namespace objects. Its protocol transitions are pure state morphisms with explicit action evidence. Its privacy protocols use separate cryptographic carriers. Its evaluation artifacts are tables, figures, algorithms, invariants, and obligations tied to the same model.

References

- [1] Pamela Zave. Reasoning about identifier spaces: How to make chord correct. *IEEE Transactions on Software Engineering*, 43(12):1144–1156, 2017. <https://arxiv.org/abs/1610.01140>.
- [2] I. Stoica, R. Morris, and D. Karger. Chord: A scalable peer-to-peer lookup service for internet applications. <https://dl.acm.org/doi/10.1145/383059.383071>, 2001.
- [3] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *Stabilization, Safety, and Security of Distributed Systems*, volume 6976 of *Lecture Notes in Computer Science*, pages 386–400. Springer, 2011.
- [4] A. Shamir. How to share a secret. <https://link.springer.com/article/10.1007/BF00185039>, 1979.
- [5] B. Campbell, R. Mahy, and C. Jennings. The message session relay protocol (msrp). RFC 4975, 2007. <https://www.rfc-editor.org/info/rfc4975>.
- [6] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.
- [7] Jonathan Nadal. Stateright: Model checking for implementing distributed systems, 2026. <https://www.stateright.rs/>.
- [8] Petar Maymounkov and David Mazières. Kademlia: A peer-to-peer information system based on the xor metric. *International Workshop on Peer-to-Peer Systems*, pages 53–65, 2002.
- [9] Juan Benet. Interplanetary file system: A p2p file system for the next web. *Proceedings of the 24th International Conference on World Wide Web*, pages 1149–1160, 2015.
- [10] J. R. Etheridge, M. Castro, and I. Stoica. Libp2p: A modular network stack for p2p systems. *USENIX Association Conference on Networked Systems Design and Implementation*, 14:1–15, 2017.
- [11] Roger Dingledine, Nick Mathewson, and Paul Syverson. Tor: The second-generation onion router. *ACM Transactions on Computer Systems*, 22(3):303–327, 2004.
- [12] Nym Development Team. Nym: A decentralized mix network. *Journal of Privacy and Security*, 6(2):135–144, 2021.